第 5 讲：队列、双端队列与 BFS

0. 本讲的核心问题

假设火从若干位置同时向上下左右扩散，每分钟前进一格。怎样计算每个位置最早在第几分钟被烧到？

这个问题的关键不是“递归”或“枚举所有路径”，而是：

> 距离起点近的状态必须先处理，距离远的状态后处理。

这种先到先处理的规则就是队列；按照距离一层层扩展的搜索就是广度优先搜索 BFS。

---

1. 队列是什么

队列遵循：

```
First In, First Out
```

基本操作：

- `enqueue(x)`：从队尾加入；
- `dequeue()`：从队首取出；
- `front()`：查看队首；
- `is_empty()`：判断是否为空。

状态示意：

```
A  [A]
B  [A, B]
C  [A, B, C]
   [B, C]       # A
```

栈处理最近加入的任务，队列处理最早等待的任务。

---

2. Python 为什么使用 `deque`

使用普通列表：

```
[python]
queue = []
queue.append(x)
x = queue.pop(0)
```

虽然逻辑正确，但 `pop(0)` 会让后面的元素整体前移，时间是 `O(n)`。

应使用：

```
[python]
from collections import deque

queue = deque()
queue.append(10)       #
queue.append(20)
x = queue.popleft()    #         10
```

`deque` 是双端队列，两端加入和删除通常都是 `O(1)`：

```
[python]
queue.append(x)
queue.appendleft(x)
queue.pop()
queue.popleft()
```

但 `deque` 不适合频繁通过中间下标随机访问。数组擅长随机访问，双端队列擅长两端操作。

---

3. 用类理解队列接口

```
[python]
from collections import deque

class Queue:
    def __init__(self):
        self._items = deque()

    def enqueue(self, value):
        self._items.append(value)

    def dequeue(self):
        if not self._items:
            raise IndexError("dequeue from empty queue")
        return self._items.popleft()

    def front(self):
        if not self._items:
            raise IndexError("front from empty queue")
        return self._items[0]

    def is_empty(self):
        return not self._items
```

在 LeetCode 中通常直接使用 `deque`，不需要每题重新封装类。

---

4. BFS 的基本思想

给定一个起点：

```
0
1
2
3
```

BFS 使用队列保证距离较小的状态先被处理：

```
[python]
from collections import deque

def bfs(start):
    queue = deque([start])
    visited = {start}

    while queue:
        current = queue.popleft()

        for neighbor in neighbors(current):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
```

核心不变量：

> 队列中状态按它们被发现的先后顺序排列；在无权图中，状态第一次被发现时，所用步数最少。

5. 为什么访问标记要在入队时设置

推荐写法：

```python
if neighbor not in visited:
    visited.add(neighbor)
    queue.append(neighbor)
```

不要等到出队时才标记。假设 `A` 和 `B` 都连接到 `C`：

```
A —> C <— B
```

如果 `C` 入队时没有标记，处理 `A` 和 `B` 时可能把 `C` 重复加入队列。

入队即标记表示：

> 一个状态一旦被安排处理，就不再重复安排。

6. BFS 求无权图最短距离

图用邻接表表示：

```python
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C"],
}
```

计算起点到各点的最短边数：

```python
from collections import deque

def shortest_distances(graph, start):
    queue = deque([start])
    distance = {start: 0}

    while queue:
        node = queue.popleft()

        for neighbor in graph[node]:
            if neighbor not in distance:
                distance[neighbor] = distance[node] + 1
                queue.append(neighbor)

    return distance
```

这里 `distance` 同时承担两个作用：

- 判断是否访问过；
- 保存最短距离。

因此不需要额外的 `visited`。

6.1 为什么第一次发现就是最短距离

BFS 先处理距离 `d` 的所有节点，再处理距离 `d+1` 的节点。一个节点第一次被发现时，不可能存在尚未处理的更短路径，否则它应该在更早的一层已经被发现。

这个结论只直接适用于每条边代价相同的情况。带不同权重的最短路不能机械使用普通 BFS。

---

7. 怎样按层处理

有时不仅需要遍历，还需要知道当前层。

```
[python]
from collections import deque

def bfs_by_level(start):
    queue = deque([start])

    while queue:
        level_size = len(queue)
        current_level = []

        for _ in range(level_size):
            node = queue.popleft()
            current_level.append(node)

            for neighbor in neighbors(node):
                queue.append(neighbor)

        print(current_level)
```

必须在处理本层之前保存：

```
[python]
level_size = len(queue)
```

因为处理过程中还会把下一层加入队列。当前这 `level_size` 个元素恰好属于同一层。

二叉树层序遍历会使用这个模板。

---

8. 网格也是图

二维网格中的每个格子可以看作节点，上下左右相邻格子是边。

```
[python]
directions = [
    (-1, 0),
    (1, 0),
    (0, -1),
    (0, 1),
]
```

从 `(row, col)` 得到相邻坐标：

```
[python]
for dr, dc in directions:
    nr = row + dr
    nc = col + dc
```

边界检查：

```
[python]
0 <= nr < rows and 0 <= nc < cols
```

8.1 网格最短路模板

```
[python]
from collections import deque

def shortest_path(grid, start, target):
    rows = len(grid)
    cols = len(grid[0])
    queue = deque([(start[0], start[1], 0)])
    visited = {start}
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while queue:
        row, col, distance = queue.popleft()

        if (row, col) == target:
            return distance

        for dr, dc in directions:
            nr = row + dr
            nc = col + dc

            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            if grid[nr][nc] == 1:
                continue
            if (nr, nc) in visited:
                continue

            visited.add((nr, nc))
            queue.append((nr, nc, distance + 1))

    return -1
```

这里假设 `1` 是障碍，无法到达时返回 `-1`。

---

9. 多源 BFS

如果火从多个位置同时开始扩散，不应该分别运行多次 BFS。可以把所有起点一开始同时入队：

```
[python]
queue = deque()

for source in sources:
    queue.append(source)
    visited.add(source)
```

之后普通 BFS 就会自然按照“距离最近的任意起点”扩散。

9.1 腐烂的橘子

网格中：

- `0`：空格；
- `1`：新鲜橘子；
- `2`：腐烂橘子。

每分钟腐烂橘子感染相邻新鲜橘子：

```
[python]
from collections import deque

def oranges_rotting(grid):
    rows = len(grid)
```

```
    cols = len(grid[0])
    queue = deque()
    fresh = 0

    for row in range(rows):
        for col in range(cols):
            if grid[row][col] == 2:
                queue.append((row, col, 0))
            elif grid[row][col] == 1:
                fresh += 1

    minutes = 0
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while queue:
        row, col, minute = queue.popleft()
        minutes = max(minutes, minute)

        for dr, dc in directions:
            nr = row + dr
            nc = col + dc

            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                grid[nr][nc] = 2
                fresh -= 1
                queue.append((nr, nc, minute + 1))

    return minutes if fresh == 0 else -1
```

把新鲜橘子改成 `2` 就同时完成了访问标记，避免重复入队。

---

10. BFS 的复杂度

图有 `V` 个节点、`E` 条边。使用邻接表时：

- 每个节点最多入队一次；
- 每条边最多被检查固定次数。

因此：

```
O(V + E)
O(V)
```

对于 `rows × cols` 网格，节点数是 `rows × cols`，每格只有常数个邻居：

```
O(rows × cols)
O(rows × cols)
```

---

11. BFS 与 DFS 的区别

特征	BFS	DFS
核心结构	队列	栈或递归
搜索方式	一层层扩展	沿一条路径深入
无权最短路	天然适合	通常不能直接保证
内存特点	可能保存整层	主要保存当前路径
常见题型	最少步数、层序、扩散	连通性、路径、回溯

如果只问“能否到达”，两者都可能适用；如果问“最少几步”，优先考虑 BFS。

---

12. 常见错误

12.1 使用 `list.pop(0)`

逻辑正确但可能把整体复杂度拖慢。使用 `deque.popleft()`。

12.2 出队时才标记访问

可能让同一状态重复入队。通常应入队时标记。

12.3 忘记边界检查顺序

必须先确认坐标合法，再读取 `grid[nr][nc]`。

12.4 混淆当前层与下一层

按层处理时，先保存 `level_size`。

12.5 对带权图机械使用 BFS

普通 BFS 保证的是边数最少，或每条边权重相同情况下的最短距离。

12.6 忘记空网格约束

访问 `grid[0]` 前，要根据题目保证或自行处理空输入。

---

13. 本讲检查表

```
1.
2.
3.
4.
5.
6.
7.
8.              BFS
9.
10.
```

14. 本讲总结

1. 队列遵循先进先出，Python 使用 `deque` 实现高效两端操作。
2. BFS 用队列按照距离由近到远扩展。
3. 无权图中，节点第一次被发现时通常得到最短距离。
4. 已访问状态通常在入队时标记。
5. 多个起点同时扩散时，把全部起点一开始入队。
6. 图 BFS 时间通常是 `O(V+E)`，网格 BFS 通常是 `O(rows×cols)`。